TP n°4

Exercice 1:: Procédures anonymes et fonctions

Question 1 : Somme de deux entiers saisis par l'utilisateur

Le bloc ci-dessous lit deux entiers et affiche leur somme. Dans SQL*Plus / SqlDeveloper, & déclenche la saisie interactive.

```
SQL> SET SERVEROUTPUT ON;
SQL> ACCEPT v_a PROMPT 'Entrez le premier entier : '
Entrez le premier entier : ACCEPT v_b PROMPT 'Entrez le deuxième entier : '
SQL>
SQL> DECLARE
  2      nb1   NUMBER := &v_a;
  3      nb2   NUMBER := &v_b;
  4      somme NUMBER;
  5  BEGIN
  6      somme := nb1 + nb2;
  7      DBMS_OUTPUT.PUT_LINE('Somme de ' || nb1 || ' et ' || nb2 || ' = ' || somme);
  8  END;
  9  /
```

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      nb1   NUMBER := 5;
  3      nb2   NUMBER := 3;
  4      somme NUMBER;
  5  BEGIN
  6      somme := nb1 + nb2;
  7      DBMS_OUTPUT.PUT_LINE('Somme = ' || somme);
  8  END;
  9  /
Somme = 8
```

Question 2 : Table de multiplication d'un nombre saisi

La boucle FOR parcourt les multiplicateurs de 1 à 10 et affiche chaque ligne de la table.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      v_n NUMBER := 7;
  3  BEGIN
  4      DBMS_OUTPUT.PUT_LINE('Table de multiplication de ' || v_n || ' :');
  5      FOR i IN 1..10 LOOP
  6          DBMS_OUTPUT.PUT_LINE(v_n || ' x ' || i || ' = ' || (v_n * i));
  7      END LOOP;
  8  END;
  9  /
Table de multiplication de 7 :
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

ProcÚdure PL/SQL terminÚe avec succÈs.
```

Question 3 : Fonction récursive x^n

La fonction PUISSANCE appelle récursivement avec n-1 jusqu'au cas de base (n=0 → retourne 1). x et n sont deux entiers positifs.

```
SQL> SET SERVEROUTPUT ON;
SQL> CREATE OR REPLACE FUNCTION PUISSANCE(p_x IN NUMBER, p_n IN NUMBER)
  2  RETURN NUMBER IS
  3  BEGIN
  4      IF p_n = 0 THEN
  5          RETURN 1;
  6      ELSE
  7          RETURN p_x * PUISSANCE(p_x, p_n - 1);
  8      END IF;
  9  END PUISSANCE;
 10  /

Fonction crÚÚe.

SQL>
SQL> -- Test de la fonction
SQL> BEGIN
  2      DBMS_OUTPUT.PUT_LINE('2^10 = ' || PUISSANCE(2, 10));
  3      DBMS_OUTPUT.PUT_LINE('3^4  = ' || PUISSANCE(3, 4));
  4  END;
  5  /
2^10 = 1024
3^4  = 81

ProcÚdure PL/SQL terminÚe avec succÈs.
```

Question 4 : Factorielle d'un nombre (stockée dans resultatFactoriel)

On crée d'abord la table resultatFactoriel, puis un bloc anonyme calcule la factorielle du nombre saisi et l'insère.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      v_n        NUMBER := 5;
  3      v_resultat NUMBER := 1;
  4  BEGIN
  5      IF v_n <= 0 THEN
  6          DBMS_OUTPUT.PUT_LINE('Erreur : le nombre doit être strictement positif.');
  7      ELSE
  8          FOR i IN 1..v_n LOOP
  9              v_resultat := v_resultat * i;
 10          END LOOP;
 11          INSERT INTO resultatFactoriel (nombre, factorielle)
 12          VALUES (v_n, v_resultat);
 13          COMMIT;
 14          DBMS_OUTPUT.PUT_LINE(v_n || '! = ' || v_resultat);
 15      END IF;
 16  END;
 17  /
5! = 120

ProcÚdure PL/SQL terminÚe avec succÞs.
```

Question 5 : Factorielles des 20 premiers entiers (stockées dans resultatsFactoriels)

On crée une table resultatsFactoriels (notez le 's'), puis une boucle FOR calcule et insère les 20 factorielles.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      v_fact NUMBER := 1;
  3  BEGIN
  4      FOR i IN 1..20 LOOP
  5          v_fact := v_fact * i;
  6          INSERT INTO resultatsFactoriels (nombre, factorielle)
  7          VALUES (i, v_fact);
  8          DBMS_OUTPUT.PUT_LINE(i || '! = ' || v_fact);
  9      END LOOP;
 10      COMMIT;
 11  END;
 12  /
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
11! = 39916800
12! = 479001600
13! = 6227020800
14! = 87178291200
15! = 1307674368000
16! = 20922789888000
17! = 355687428096000
18! = 6402373705728000
19! = 121645100408832000
20! = 2432902008176640000

ProcÚdure PL/SQL terminÚe avec succPs.
```

Exercice 2 : Gestion des employés (table EMP)

Création de la table EMP et insertion de données

Commencer par créer et peupler la table :

```
SQL> CREATE TABLE emp (
  2      matr    NUMBER(10)    NOT NULL,
  3      nom     VARCHAR2(50)  NOT NULL,
  4      sal     NUMBER(7,2),
  5      adresse VARCHAR2(96),
  6      dep     NUMBER(10)    NOT NULL,
  7      CONSTRAINT emp_pk PRIMARY KEY (matr)
  8  );

Table crÚÚe.
```

```
SQL>
SQL> INSERT INTO emp VALUES (1, 'Alice',   3500, '12 rue de Rivoli', 92000);

1 ligne crÚÚe.

SQL> INSERT INTO emp VALUES (2, 'Bob',     2800, '5 avenue Victor Hugo', 75000);

1 ligne crÚÚe.

SQL> INSERT INTO emp VALUES (3, 'Clara',   4200, '7 bd Saint-Germain', 92000);

1 ligne crÚÚe.

SQL> INSERT INTO emp VALUES (4, 'David',   3100, '18 rue Lafayette', 75000);

1 ligne crÚÚe.

SQL> INSERT INTO emp VALUES (5, 'Emma',    2600, '3 rue de la Paix', 92000);

1 ligne crÚÚe.

SQL> INSERT INTO emp VALUES (6, 'Farida',  3800, '22 avenue Montaigne', 10);

1 ligne crÚÚe.

SQL> COMMIT;

Validation effectuÚe.
```

Question 1 : Insérer un employé avec %ROWTYPE

On déclare une variable de type EMP%ROWTYPE, on renseigne chaque champ, puis on insère le n-uplet en une seule instruction.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2       v_employe emp%ROWTYPE;
  3   BEGIN
  4       v_employe.matr    := 7;
  5       v_employe.nom     := 'Youcef';
  6       v_employe.sal     := 2500;
  7       v_employe.adresse := 'avenue de la République';
  8       v_employe.dep     := 92002;
  9       INSERT INTO emp VALUES v_employe;
 10       COMMIT;
 11       DBMS_OUTPUT.PUT_LINE('Employé inséré avec succès.');
 12   END;
 13   /
Employé inséré avec succès.

ProcÚdure PL/SQL terminÚe avec succPs.
```

Question 2 : Supprimer des employés et afficher le nombre de lignes supprimées

SQL%ROWCOUNT contient le nombre de lignes affectées par le dernier DML. Ici on supprime les employés du département 10.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      v_nb_lignes NUMBER;
  3  BEGIN
  4      DELETE FROM emp WHERE dep = 10;
  5      v_nb_lignes := SQL%ROWCOUNT;
  6      DBMS_OUTPUT.PUT_LINE('Nombre de lignes supprimées : ' || v_nb_lignes);
  7      COMMIT;
  8  END;
  9  /
Nombre de lignes supprimées : 1

ProcÚdure PL/SQL terminÚe avec succÞs.
```

Question 3 : Somme des salaires avec curseur explicite et LOOP / EXIT WHEN

On ouvre manuellement le curseur, on récupère les lignes une à une avec FETCH, et on sort quand %NOTFOUND est vrai. Équivalent à SELECT SUM(sal) FROM emp.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      v_salaire emp.sal%TYPE;
  3      v_total   emp.sal%TYPE := 0;
  4      CURSOR c_salaires IS
  5          SELECT sal FROM emp;
  6  BEGIN
  7      OPEN c_salaires;
  8      LOOP
  9          FETCH c_salaires INTO v_salaire;
 10          EXIT WHEN c_salaires%NOTFOUND;
 11          IF v_salaire IS NOT NULL THEN
 12              v_total := v_total + v_salaire;
 13          END IF;
 14      END LOOP;
 15      CLOSE c_salaires;
 16      DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
 17  END;
 18  /
Somme des salaires : 18700

ProcÚdure PL/SQL terminÚe avec succÞs.
```

Question 4 : Salaire moyen avec curseur explicite

On adapte la question 3 : on compte les employés avec v_count et on divise la somme par ce compteur à la fin. Équivalent à SELECT AVG(sal) FROM emp.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      v_salaire emp.sal%TYPE;
  3      v_total   emp.sal%TYPE := 0;
  4      v_count   NUMBER       := 0;
  5      CURSOR c_salaires IS
  6          SELECT sal FROM emp;
  7  BEGIN
  8      OPEN c_salaires;
  9      LOOP
 10          FETCH c_salaires INTO v_salaire;
 11          EXIT WHEN c_salaires%NOTFOUND;
 12          IF v_salaire IS NOT NULL THEN
 13              v_total := v_total + v_salaire;
 14              v_count := v_count + 1;
 15          END IF;
 16      END LOOP;
 17      CLOSE c_salaires;
 18      IF v_count > 0 THEN
 19          DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || ROUND(v_total / v_count, 2));
 20      ELSE
 21          DBMS_OUTPUT.PUT_LINE('Aucun employé trouvé.');
 22      END IF;
 23  END;
 24  /
Salaire moyen : 3116,67

ProcÚdure PL/SQL terminÚe avec succÞs.
```

Question 5 : Version avec boucle FOR IN (somme et moyenne)

La boucle FOR IN ouvre, lit et ferme le curseur automatiquement. Pas besoin de OPEN/FETCH/CLOSE.

Somme des salaires — version FOR IN :

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      v_total emp.sal%TYPE := 0;
  3      CURSOR c_salaires IS
  4          SELECT sal FROM emp;
  5  BEGIN
  6      FOR v_ligne IN c_salaires LOOP
  7          IF v_ligne.sal IS NOT NULL THEN
  8              v_total := v_total + v_ligne.sal;
  9          END IF;
 10      END LOOP;
 11      DBMS_OUTPUT.PUT_LINE('Somme des salaires : ' || v_total);
 12  END;
 13  /
Somme des salaires : 18700

ProcÚdure PL/SQL terminÚe avec succÞs.
```

Salaire moyen — version FOR IN :

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      v_total emp.sal%TYPE := 0;
  3      v_count NUMBER       := 0;
  4      CURSOR c_salaires IS
  5          SELECT sal FROM emp;
  6  BEGIN
  7      FOR v_ligne IN c_salaires LOOP
  8          IF v_ligne.sal IS NOT NULL THEN
  9              v_total := v_total + v_ligne.sal;
 10              v_count := v_count + 1;
 11          END IF;
 12      END LOOP;
 13      IF v_count > 0 THEN
 14          DBMS_OUTPUT.PUT_LINE('Salaire moyen : ' || ROUND(v_total / v_count, 2));
 15      ELSE
 16          DBMS_OUTPUT.PUT_LINE('Aucun employé.');
 17      END IF;
 18  END;
 19  /
Salaire moyen : 3116,67

ProcÚdure PL/SQL terminÚe avec succPs.
```

Question 6 : Curseur paramétré pour deux départements

Le curseur prend un paramètre p_dep. On l'appelle deux fois avec des valeurs différentes (92000 et 75000) dans deux boucles FOR IN distinctes.

```
SQL> SET SERVEROUTPUT ON;
SQL> DECLARE
  2      CURSOR c(p_dep emp.dep%TYPE) IS
  3          SELECT dep, nom
  4          FROM emp
  5          WHERE dep = p_dep;
  6  BEGIN
  7      DBMS_OUTPUT.PUT_LINE('=== Département 92000 ===');
  8      FOR v_employe IN c(92000) LOOP
  9          DBMS_OUTPUT.PUT_LINE('  Nom : ' || v_employe.nom);
 10      END LOOP;
 11
 12      DBMS_OUTPUT.PUT_LINE('=== Département 75000 ===');
 13      FOR v_employe IN c(75000) LOOP
 14          DBMS_OUTPUT.PUT_LINE('  Nom : ' || v_employe.nom);
 15      END LOOP;
 16  END;
 17  /
=== Département 92000 ===
Nom : Alice
Nom : Clara
Nom : Emma
=== Département 75000 ===
Nom : Bob
Nom : David

ProcÚdure PL/SQL terminÚe avec succPs.
```

Exercice 3 : Package de gestion des clients (surcharge + exceptions)

On définit un package PKG_CLIENT avec deux procédures surchargées AJOUTER_CLIENT : l'une prend un n-uplet complet (%ROWTYPE), l'autre accepte les paramètres séparément. Toutes les exceptions Oracle sont levées explicitement.

Création de la table CLIENT

```
SQL> CREATE TABLE client (
  2      id_client NUMBER(10)   NOT NULL,
  3      nom       VARCHAR2(50) NOT NULL,
  4      email     VARCHAR2(100),
  5      tel       VARCHAR2(20),
  6      CONSTRAINT client_pk PRIMARY KEY (id_client)
  7  );

Table crÚÚe.
```

Spécification du package PKG_CLIENT

```
SQL> CREATE OR REPLACE PACKAGE PKG_CLIENT AS
  2
  3      -- Surcharge 1 : insertion via un n-uplet complet
  4      PROCEDURE AJOUTER_CLIENT(p_client IN client%ROWTYPE);
  5
  6      -- Surcharge 2 : insertion par paramètres séparés
  7      PROCEDURE AJOUTER_CLIENT(
  8          p_id    IN client.id_client%TYPE,
  9          p_nom   IN client.nom%TYPE,
 10          p_email IN client.email%TYPE DEFAULT NULL,
 11          p_tel   IN client.tel%TYPE   DEFAULT NULL
 12      );
 13
 14  END PKG_CLIENT;
 15  /

Package crÚÚ.
```

Corps du package PKG_CLIENT

```
SQL> CREATE OR REPLACE PACKAGE BODY PKG_CLIENT AS
  2
  3     PROCEDURE AJOUTER_CLIENT(p_client IN client%ROWTYPE) IS
  4     BEGIN
  5         IF p_client.id_client IS NULL THEN
  6             RAISE_APPLICATION_ERROR(-20001, 'L''id_client ne peut pas être NULL.');
  7         END IF;
  8
  9         IF p_client.nom IS NULL OR TRIM(p_client.nom) = '' THEN
 10             RAISE_APPLICATION_ERROR(-20002, 'Le nom du client est obligatoire.');
 11         END IF;
 12
 13         INSERT INTO client VALUES p_client;
 14         COMMIT;
 15         DBMS_OUTPUT.PUT_LINE('Client ajouté (surcharge 1) : ' || p_client.nom);
 16
 17     EXCEPTION
 18         WHEN DUP_VAL_ON_INDEX THEN
 19             ROLLBACK;
 20             DBMS_OUTPUT.PUT_LINE('Erreur : un client avec l''id '
 21                 || p_client.id_client || ' existe déjà.');
 22         WHEN OTHERS THEN
 23             ROLLBACK;
 24             DBMS_OUTPUT.PUT_LINE('Erreur inattendue (S1) : ' || SQLERRM);
 25     END AJOUTER_CLIENT;
 26
 27     PROCEDURE AJOUTER_CLIENT(
 28         p_id    IN client.id_client%TYPE,
 29         p_nom   IN client.nom%TYPE,
 30         p_email IN client.email%TYPE DEFAULT NULL,
 31         p_tel   IN client.tel%TYPE   DEFAULT NULL
 32     ) IS
 33         v_client client%ROWTYPE;
 34     BEGIN
 35         IF p_id IS NULL THEN
 36             RAISE_APPLICATION_ERROR(-20001, 'L''id_client ne peut pas être NULL.');
 37         END IF;
 38
 39         IF p_nom IS NULL OR TRIM(p_nom) = '' THEN
 40             RAISE_APPLICATION_ERROR(-20002, 'Le nom du client est obligatoire.');
 41         END IF;
 42
 43         v_client.id_client := p_id;
 44         v_client.nom       := p_nom;
 45         v_client.email     := p_email;
 46         v_client.tel       := p_tel;
 47
 48         INSERT INTO client VALUES v_client;
 49         COMMIT;
 50         DBMS_OUTPUT.PUT_LINE('Client ajouté (surcharge 2) : ' || p_nom);
 51
 52     EXCEPTION
 53         WHEN DUP_VAL_ON_INDEX THEN
 54             ROLLBACK;
 55             DBMS_OUTPUT.PUT_LINE('Erreur : un client avec l''id '
 56                 || p_id || ' existe déjà.');
 57         WHEN OTHERS THEN
 58             ROLLBACK;
 59             DBMS_OUTPUT.PUT_LINE('Erreur inattendue (S2) : ' || SQLERRM);
 60     END AJOUTER_CLIENT;
 61
```

```
 61
 62  END PKG_CLIENT;
 63  /

Corps de package crÚÚ.
```

Tests du package

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> DECLARE
  2      v_c client%ROWTYPE;
  3  BEGIN
  4      v_c.id_client := 1;
  5      v_c.nom       := 'Marie Dupont';
  6      v_c.email     := 'marie@example.com';
  7      v_c.tel       := '0612345678';
  8      PKG_CLIENT.AJOUTER_CLIENT(v_c);
  9  END;
 10  /
Client ajouté (surcharge 1) : Marie Dupont

ProcÚdure PL/SQL terminÚe avec succÞs.

SQL>
SQL> BEGIN
  2      PKG_CLIENT.AJOUTER_CLIENT(2, 'Jean Martin', 'jean@example.com');
  3      PKG_CLIENT.AJOUTER_CLIENT(3, 'Sara Ben Ali');
  4  END;
  5  /
Client ajouté (surcharge 2) : Jean Martin
Client ajouté (surcharge 2) : Sara Ben Ali

ProcÚdure PL/SQL terminÚe avec succÞs.

SQL>
SQL> BEGIN
  2      PKG_CLIENT.AJOUTER_CLIENT(1, 'Doublon Test');
  3  END;
  4  /
Erreur : un client avec l'id 1 existe déjà.

ProcÚdure PL/SQL terminÚe avec succÞs.

SQL>
SQL> BEGIN
  2      PKG_CLIENT.AJOUTER_CLIENT(99, NULL);
  3  END;
  4  /
Erreur inattendue (S2) : ORA-20002: Le nom du client est obligatoire.

ProcÚdure PL/SQL terminÚe avec succÞs.

SQL>
```